

07_migraciones-alembic

#SQLAlchemy #PostgreSQL #python #alembic

1 Ubicación dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ alembic/  
        └─ versions/  
            ├── c031417b68f1_init.py  
            ├── b8f4b712e6b5_create_events_table.py  
            ├── be0f61d66ed2_add_meta_to_events.py  
            ├── d841bcb4d197_add_rules_and_alerts.py  
            ├── cbd8e2a0c1fe_add_throttle_seconds_to_rules.py  
            ├── 3099c4ee7f79_add_throttle_to_rules.py  
            ├── b1b85630457f_add_threshold_to_rules.py  
            ├── 41bf261af532_add_group_key_to_alerts.py  
            ├── 2e15d22277a_add_status_and_updated_at_to_alerts.py  
            └─ d7f85cce3934_fix_group_key_default.py
```

Las migraciones de Alembic se encuentran dentro de:

```
backend/alembic/versions/
```

Cada archivo representa un cambio concreto en la estructura de la base de datos.

Mientras que los modelos SQLAlchemy definen cómo debería ser la estructura desde el código Python, las migraciones indican cómo aplicar esos cambios físicamente sobre PostgreSQL.

La relación general es:

```
Modelos SQLAlchemy  
    ↓  
Alembic  
    ↓  
Migraciones  
    ↓  
Tablas reales en PostgreSQL
```

2 Comandos utilizados para visualizar las migraciones

Para listar las migraciones:

```
cd ~/siem-lab  
ls -l backend/alembic/versions
```

Para ver el contenido de todas las migraciones:

```
cd ~/siem-lab  
  
for file in backend/alembic/versions/*.py; do  
    echo "==== $file ==="  
    sed -n '1,220p' "$file"  
    echo  
done
```

El primer comando muestra todos los archivos de migración existentes.

El segundo comando recorre cada archivo `.py` dentro de `backend/alembic/versions/` y muestra su contenido.

3 Función general de Alembic en el proyecto

Alembic se utiliza para controlar la evolución de la base de datos.

En este proyecto, la base de datos no se crea manualmente desde Adminer ni escribiendo SQL directamente a mano. En su lugar, se usan migraciones.

Una migración sirve para aplicar cambios como:

- Crear tablas.
- Añadir columnas.
- Modificar columnas existentes.
- Crear índices.
- Eliminar columnas al hacer downgrade.
- Revertir cambios si fuera necesario.

En este laboratorio, las migraciones permiten crear y evolucionar las tablas principales:

```
events
rules
alerts
```

Estas tablas corresponden a los modelos analizados previamente:

```
Event → events
Rule  → rules
Alert → alerts
```

4 Orden lógico de las migraciones

Aunque el listado alfabético no siempre muestra el orden funcional de forma evidente, Alembic usa los campos `revision` y `down_revision` para encadenar las migraciones.

El orden lógico del proyecto es:

1. c031417b68f1_init
2. b8f4b712e6b5_create_events_table
3. be0f61d66ed2_add_meta_to_events
4. d841bcb4d197_add_rules_and_alerts
5. cbd8e2a0c1fe_add_throttle_seconds_to_rules
6. 3099c4ee7f79_add_throttle_to_rules
7. b1b85630457f_add_threshold_to_rules
8. 41bf261af532_add_group_key_to_alerts
9. 2e15d222277a_add_status_and_updated_at_to_alerts
10. d7f85cce3934_fix_group_key_default

La cadena se construye así:

```
c031417b68f1
↓
b8f4b712e6b5
↓
be0f61d66ed2
↓
d841bcb4d197
↓
cbd8e2a0c1fe
↓
3099c4ee7f79
↓
b1b85630457f
↓
41bf261af532
```

```
↓
2e15d222277a
↓
d7f85cce3934
```

Cada migración conoce cuál es la anterior mediante:

```
down_revision = "id_de_la_migracion_anterior"
```

5 Estructura común de una migración Alembic

La mayoría de migraciones siguen una estructura parecida:

```
"""descripción de la migración

Revision ID: ...
Revises: ...
Create Date: ...

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa

revision: str = "..."
down_revision: Union[str, Sequence[str], None] = "..."
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None


def upgrade() -> None:
    ...


def downgrade() -> None:
    ...
```

Los elementos principales son:

revision	→ identificador único de la migración
down_revision	→ migración anterior
upgrade()	→ cambios que se aplican hacia adelante
downgrade()	→ cambios para revertir la migración

6 Análisis de cada migración

6.1 Migración inicial: c031417b68f1_init.py

```
revision: str = 'c031417b68f1'
down_revision: Union[str, Sequence[str], None] = None
```

Esta es la migración inicial.

El campo:

```
down_revision = None
```

indica que no tiene ninguna migración anterior.

Es el punto de inicio de la cadena de Alembic.

El contenido de `upgrade()` es:

```
def upgrade() -> None:
    """Upgrade schema."""
    pass
```

Y el de `downgrade()`:

```
def downgrade() -> None:
    """Downgrade schema."""
    pass
```

Esto significa que esta migración no crea ninguna tabla ni modifica la base de datos.

Su función es actuar como punto inicial del historial de migraciones.

Conceptualmente:

```
c031417b68f1_init
    ↓
punto de partida del esquema
```

6.2 Creación de tabla `events` : `b8f4b712e6b5_create_events_table.py`

Esta migración crea la tabla inicial de eventos.

```
revision: str = 'b8f4b712e6b5'
down_revision: Union[str, Sequence[str], None] = 'c031417b68f1'
```

Esto indica que viene después de la migración inicial.

Función `upgrade`

```
op.create_table('events',
sa.Column('id', sa.Integer(), nullable=False),
sa.Column('ts', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
sa.Column('source', sa.String(length=64), nullable=False),
sa.Column('severity', sa.Integer(), nullable=False),
sa.Column('message', sa.Text(), nullable=False),
sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
sa.PrimaryKeyConstraint('id')
)
```

Esta instrucción crea la tabla:

```
events
```

con las columnas:

```
id
ts
source
severity
message
created_at
```

Relación con el modelo:

```
backend/app/models/event.py
```

La tabla `events` representa los eventos recibidos por el laboratorio SIEM.

Columnas creadas

```
id          → clave primaria
ts          → timestamp del evento
source      → origen del evento
severity    → severidad
message     → mensaje descriptivo
created_at  → fecha de creación en base de datos
```

Función downgrade

```
op.drop_table('events')
```

Si se revierte esta migración, se elimina la tabla `events`.

6.3 Añadir meta a eventos: `be0f61d66ed2_add_meta_to_events.py`

Esta migración añade el campo `meta` a la tabla `events`.

```
revision: str = 'be0f61d66ed2'
down_revision: Union[str, Sequence[str], None] = 'b8f4b712e6b5'
```

Viene después de la creación de la tabla `events`.

Función upgrade

```
op.add_column('events', sa.Column('meta', postgresql.JSONB(astext_type=sa.Text()), nullable=True))
```

Esta línea añade una columna nueva:

```
meta
```

a la tabla:

```
events
```

El tipo es:

```
JSONB
```

Esto permite almacenar metadatos flexibles asociados al evento.

Ejemplo:

```
{
  "ip": "192.168.1.10",
  "user": "admin",
  "action": "login_failed"
}
```

Este cambio se corresponde con el campo del modelo `Event`:

```
meta: Mapped[Optional[dict[str, Any]]] = mapped_column(JSONB, nullable=True)
```

Función downgrade

```
op.drop_column('events', 'meta')
```

Si se revierte la migración, se elimina la columna `meta`.

6.4 Crear reglas y alertas: `d841bcb4d197_add_rules_and_alerts.py`

Esta migración crea las tablas `rules` y `alerts`.

```
revision: str = 'd841bcb4d197'
down_revision: Union[str, Sequence[str], None] = 'be0f61d66ed2'
```

Viene después de añadir `meta` a eventos.

Creación de tabla `rules`

```
op.create_table('rules',
sa.Column('id', sa.Integer(), nullable=False),
sa.Column('name', sa.String(length=120), nullable=False),
sa.Column('enabled', sa.Boolean(), server_default='true', nullable=False),
sa.Column('source', sa.String(length=64), nullable=True),
sa.Column('severity_min', sa.Integer(), nullable=True),
sa.Column('contains', sa.String(length=200), nullable=True),
sa.Column('meta_match', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
sa.PrimaryKeyConstraint('id'),
sa.UniqueConstraint('name')
)
```

Esta tabla almacena reglas de detección.

Columnas principales:

<code>id</code>	→ clave primaria
<code>name</code>	→ nombre de la regla
<code>enabled</code>	→ indica si está activa
<code>source</code>	→ filtro por origen
<code>severity_min</code>	→ severidad mínima
<code>contains</code>	→ texto que debe contener el mensaje
<code>meta_match</code>	→ coincidencias sobre metadatos
<code>created_at</code>	→ fecha de creación

La restricción:

```
sa.UniqueConstraint('name')
```

impide crear dos reglas con el mismo nombre.

Creación de tabla `alerts`

```
op.create_table('alerts',
sa.Column('id', sa.Integer(), nullable=False),
sa.Column('rule_id', sa.Integer(), nullable=False),
sa.Column('event_id', sa.Integer(), nullable=False),
sa.Column('title', sa.String(length=200), nullable=False),
sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
sa.ForeignKeyConstraint(['event_id'], ['events.id'], ondelete='CASCADE'),
sa.ForeignKeyConstraint(['rule_id'], ['rules.id'], ondelete='CASCADE'),
sa.PrimaryKeyConstraint('id')
)
```

Esta tabla almacena alertas generadas por reglas.

Columnas iniciales:

```
id          → clave primaria
rule_id     → regla que generó la alerta
event_id    → evento asociado
title       → título de la alerta
created_at  → fecha de creación
```

También define claves foráneas:

```
alerts.event_id → events.id
alerts.rule_id  → rules.id
```

Con:

```
ondelete='CASCADE'
```

Esto implica que, si se elimina un evento o una regla, las alertas asociadas también se eliminan.

Función downgrade

```
op.drop_table('alerts')
op.drop_table('rules')
```

Al revertir, primero se elimina `alerts` y después `rules`.

Este orden tiene sentido porque `alerts` depende de `rules` y `events`.

6.5 Añadir `throttle_seconds`: `cbd8e2a0c1fe_add_throttle_seconds_to_rules.py`

Esta migración añade un campo de throttle a la tabla `rules`.

```
revision: str = 'cbd8e2a0c1fe'
down_revision: Union[str, Sequence[str], None] = 'd841bcb4d197'
```

Función upgrade

```
op.add_column('rules', sa.Column('throttle_seconds', sa.Integer(), nullable=True))
```

Añade la columna:

```
throttle_seconds
```

a la tabla:

```
rules
```

Este campo permite definir un tiempo mínimo entre alertas generadas por la misma regla.

Ejemplo conceptual:

```
throttle_seconds = 300
```

Significa que la regla podría esperar 300 segundos antes de generar otra alerta similar.

Función downgrade

```
op.drop_column('rules', 'throttle_seconds')
```

Revierte el cambio eliminando la columna.

6.6 Migración vacía de throttle: 3099c4ee7f79_add_throttle_to_rules.py

Esta migración tiene este contenido:

```
def upgrade() -> None:
    """Upgrade schema."""
    pass
```

Y:

```
def downgrade() -> None:
    """Downgrade schema."""
    pass
```

Su down_revision es:

```
down_revision = 'cbd8e2a0c1fe'
```

Esto significa que está en la cadena de migraciones, pero no aplica cambios reales en la base de datos.

Puede ocurrir cuando se genera una migración automáticamente pero Alembic no detecta diferencias nuevas, o cuando una modificación ya había sido incluida en una migración anterior.

En este caso, el campo real de throttle ya fue añadido por:

```
cbd8e2a0c1fe_add_throttle_seconds_to_rules.py
```

Por tanto, esta migración actúa como un paso vacío dentro del historial.

No es peligrosa, pero conviene saber que no modifica el esquema.

6.7 Añadir threshold a reglas: b1b85630457f_add_threshold_to_rules.py

Esta migración añade soporte para reglas de tipo threshold.

```
revision: str = 'b1b85630457f'
down_revision: Union[str, Sequence[str], None] = '3099c4ee7f79'
```

Función upgrade

```
op.add_column('rules', sa.Column('threshold_count', sa.Integer(), nullable=True))
op.add_column('rules', sa.Column('threshold_seconds', sa.Integer(), nullable=True))
```

Añade dos columnas a la tabla rules :

```
threshold_count
threshold_seconds
```

Estas columnas permiten expresar reglas como:

```
N eventos en X segundos
```

Ejemplo:

```
threshold_count = 5
threshold_seconds = 60
```

Esto representa:

```
5 eventos en 60 segundos
```

Función downgrade

```
op.drop_column('rules', 'threshold_seconds')
op.drop_column('rules', 'threshold_count')
```

Revierte la migración eliminando ambas columnas.

6.8 Añadir group_key a alertas: 41bf261af532_add_group_key_to_alerts.py

Esta migración añade el campo `group_key` a la tabla `alerts`.

```
revision: str = '41bf261af532'
down_revision: Union[str, Sequence[str], None] = 'b1b85630457f'
```

Función upgrade

```
op.add_column('alerts', sa.Column('group_key', sa.String(length=128), server_default='',
nullable=False))
op.create_index(op.f('ix_alerts_event_id'), 'alerts', ['event_id'], unique=False)
op.create_index(op.f('ix_alerts_group_key'), 'alerts', ['group_key'], unique=False)
op.create_index(op.f('ix_alerts_rule_id'), 'alerts', ['rule_id'], unique=False)
```

Añade la columna:

```
group_key
```

a la tabla `alerts`.

Inicialmente se crea como:

```
String(128)
server_default=''
nullable=False
```

Esto significa que no podía ser nula y, si no se indicaba valor, usaba cadena vacía.

También crea índices sobre:

```
event_id
group_key
rule_id
```

Estos índices aceleran consultas frecuentes sobre alertas.

Función downgrade

```
op.drop_index(op.f('ix_alerts_rule_id'), table_name='alerts')
op.drop_index(op.f('ix_alerts_group_key'), table_name='alerts')
op.drop_index(op.f('ix_alerts_event_id'), table_name='alerts')
op.drop_column('alerts', 'group_key')
```

Revierte los cambios eliminando índices y después la columna.

6.9 Añadir estado y actualización a alertas: 2e15d222277a_add_status_and_updated_at_to_alerts.py

Esta migración amplía la tabla `alerts` con campos de ciclo de vida.

```
revision: str = '2e15d222277a'
down_revision: Union[str, Sequence[str], None] = '41bf261af532'
```

Función upgrade

```
op.add_column('alerts', sa.Column('status', sa.String(length=16), server_default='open',
nullable=False))
op.add_column('alerts', sa.Column('updated_at', sa.DateTime(timezone=True),
server_default=sa.text('now()'), nullable=False))
```

Añade las columnas:

```
status
updated_at
```

Campo status

```
status
```

Permite gestionar el ciclo de vida de una alerta:

```
open    → abierta
ack     → reconocida
closed  → cerrada
```

Tiene valor por defecto:

```
open
```

Campo updated_at

```
updated_at
```

Permite registrar la fecha de última actualización de la alerta.

Modificación de group_key

```
op.alter_column('alerts', 'group_key',
existing_type=sa.VARCHAR(length=128),
type_=sa.String(length=120),
nullable=True,
existing_server_default=sa.text("'':character varying"))
```

Esta parte cambia la columna `group_key`.

Cambios principales:

```
Antes: VARCHAR(128), NOT NULL, default ''
Después: String(120), nullable=True
```

Este cambio acerca la columna al modelo actual de `Alert`, donde `group_key` puede ser `None`.

Creación de índices compuestos

```
op.create_index('ix_alerts_group_key_created_at', 'alerts', ['group_key', 'created_at'], unique=False)
op.create_index('ix_alerts_rule_id_created_at', 'alerts', ['rule_id', 'created_at'], unique=False)
op.create_index(op.f('ix_alerts_status'), 'alerts', ['status'], unique=False)
```

Crea índices para acelerar consultas por:

```
group_key + created_at
rule_id + created_at
status
```

Estos índices son coherentes con consultas habituales:

```
- alertas por grupo
- alertas recientes por regla
- alertas por estado
```

Función downgrade

El `downgrade()` elimina los índices, revierte `group_key` y elimina `updated_at` y `status`.

6.10 Corregir valor por defecto de `group_key` : `d7f85cce3934_fix_group_key_default.py`

Esta es la última migración de la cadena.

```
revision: str = "d7f85cce3934"
down_revision: Union[str, Sequence[str], None] = "2e15d22277a"
```

Su objetivo es corregir definitivamente el comportamiento de `group_key`.

Función upgrade

```
op.alter_column(
    "alerts",
    "group_key",
    existing_type=sa.String(length=120),
    type_=sa.String(length=120),
    nullable=True,
    server_default=None,
)
```

Esta migración elimina el valor por defecto de `group_key` y permite valores nulos.

El propio comentario del archivo lo explica:

```
# Quitar default '' y permitir NULL.
# Además, alinear el tipo a VARCHAR(120) (tu modelo actual).
```

Esto deja `group_key` alineado con el modelo actual:

```
group_key: Mapped[str | None] = mapped_column(String(120), nullable=True, index=True)
```

Es decir:

```
group_key puede tener valor
o puede ser NULL
```

Esto es más correcto que usar cadena vacía como valor por defecto.

Función downgrade

```
op.execute("UPDATE alerts SET group_key = '' WHERE group_key IS NULL")
```

Antes de volver al estado anterior, convierte los valores `NULL` en cadena vacía.

Esto es necesario porque el estado anterior no permitía nulos.

Después ejecuta:

```
op.alter_column(  
    "alerts",  
    "group_key",  
    existing_type=sa.String(length=120),  
    type_=sa.String(length=120),  
    nullable=False,  
    server_default="",  
)
```

Esto restaura:

```
NOT NULL + default ''
```

7 Relación entre migraciones y modelos actuales

Los modelos actuales son:

```
Event  
Rule  
Alert
```

Y las migraciones explican cómo se llegó a ellos progresivamente.

Modelo Event

Modelo actual:

```
id  
ts  
source  
severity  
message  
created_at  
meta
```

Migraciones relacionadas:

```
b8f4b712e6b5_create_events_table  
be0f61d66ed2_add_meta_to_events
```

Primero se creó la tabla `events`, y después se añadió `meta`.

Modelo Rule

Modelo actual:

```
id  
name  
enabled  
source  
severity_min  
contains  
throttle_seconds  
threshold_count  
threshold_seconds  
meta_match  
created_at
```

Migraciones relacionadas:

```
d841bcb4d197_add_rules_and_alerts
cbd8e2a0c1fe_add_throttle_seconds_to_rules
b1b85630457f_add_threshold_to_rules
```

Primero se creó la tabla `rules`, después se añadió `throttle` y posteriormente `threshold`.

La migración:

```
3099c4ee7f79_add_throttle_to_rules
```

queda como una migración vacía dentro del historial.

Modelo `Alert`

Modelo actual:

```
id
rule_id
event_id
title
group_key
status
created_at
updated_at
```

Migraciones relacionadas:

```
d841bcb4d197_add_rules_and_alerts
41bf261af532_add_group_key_to_alerts
2e15d22277a_add_status_and_updated_at_to_alerts
d7f85cce3934_fix_group_key_default
```

Primero se creó la tabla `alerts`, después se añadió `group_key`, más adelante `status` y `updated_at`, y finalmente se corrigió el comportamiento de `group_key`.

8 Relación con el flujo técnico del laboratorio

Las migraciones no procesan eventos ni generan alertas directamente, pero preparan la estructura de base de datos necesaria para que el flujo pueda ejecutarse.

La relación técnica sería:

```
Alembic migrations
  ↓
crean tablas events, rules y alerts
  ↓
FastAPI puede guardar eventos
  ↓
FastAPI puede consultar reglas
  ↓
FastAPI puede guardar alertas
  ↓
frontend/API puede consultar los resultados
```

Dentro del flujo general del SIEM:

```
POST /ingest
  ↓
necesita tabla events
  ↓
motor de reglas
```

```
↓
necesita tabla rules
↓
generación de alertas
↓
necesita tabla alerts
```

Sin migraciones, los modelos existirían en Python, pero PostgreSQL no tendría necesariamente las tablas físicas creadas.

9 Errores típicos o puntos importantes

Migraciones no aplicadas

Si las migraciones no se han ejecutado, pueden aparecer errores como:

```
relation "events" does not exist
relation "rules" does not exist
relation "alerts" does not exist
```

Solución:

```
docker exec -it siem-api alembic upgrade head
```

Diferencia entre modelo y base de datos

Modificar un modelo Python no cambia automáticamente PostgreSQL.

Ejemplo:

```
class Event(Base):
    ...
```

Si se añade una columna al modelo, también debe existir una migración que la añada a la base de datos.

Migración vacía

La migración:

```
3099c4ee7f79_add_throttle_to_rules.py
```

no aplica cambios porque contiene:

```
pass
```

No es un error crítico, pero conviene saberlo.

En la práctica, funciona como un paso vacío dentro del historial.

group_key evolucionó en varias fases

El campo `group_key` primero se añadió como:

```
NOT NULL + default ''
```

Después se modificó para permitir nulos.

Finalmente se corrigió el valor por defecto.

Esto refleja una evolución real del diseño: al principio se forzó un valor vacío, pero el modelo final es más limpio permitiendo `NULL`.

onDelete="CASCADE"

Las migraciones de `alerts` crean claves foráneas con:

```
onDelete='CASCADE'
```

Esto implica que eliminar eventos o reglas puede eliminar alertas asociadas.

Es coherente para evitar registros huérfanos, pero hay que tenerlo presente.

Alembic usa una cadena de revisiones

No basta con mirar el nombre de archivo.

La relación real entre migraciones se define con:

```
revision
down_revision
```

Eso permite a Alembic saber qué migraciones están aplicadas y cuál es la siguiente.

10 Comandos útiles relacionados

Ver migraciones disponibles:

```
ls -l backend/alembic/versions
```

Ver migración actual aplicada:

```
docker exec -it siem-api alembic current
```

Ver historial de migraciones:

```
docker exec -it siem-api alembic history
```

Ver historial en formato detallado:

```
docker exec -it siem-api alembic history --verbose
```

Aplicar todas las migraciones pendientes:

```
docker exec -it siem-api alembic upgrade head
```

Revertir una migración:

```
docker exec -it siem-api alembic downgrade -1
```

Comprobar tablas creadas:

```
docker exec -it siem-db psql -U siem -d siem -c "\dt"
```

Ver estructura de `events`:

```
docker exec -it siem-db psql -U siem -d siem -c "\d events"
```

Ver estructura de `rules`:

```
docker exec -it siem-db psql -U siem -d siem -c "\d rules"
```

Ver estructura de `alerts`:

```
docker exec -it siem-db psql -U siem -d siem -c "\d alerts"
```

Consultar tabla interna de Alembic:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT * FROM alembic_version;"
```

Ejecutar migraciones desde la carpeta backend si se trabaja fuera de Docker:

```
cd backend  
alembic upgrade head
```

Con esto queda cerrado el módulo:

```
```text  
02_Base-de-datos
└─ 03_Analisis-tecnico-de-base-de-datos
 ├── 01_database-py
 ├── 02_session-py
 ├── 03_base-py
 ├── 04_model-event-py
 ├── 05_model-rule-py
 ├── 06_model-alert-py
 └─ 07_migraciones-alembic
```